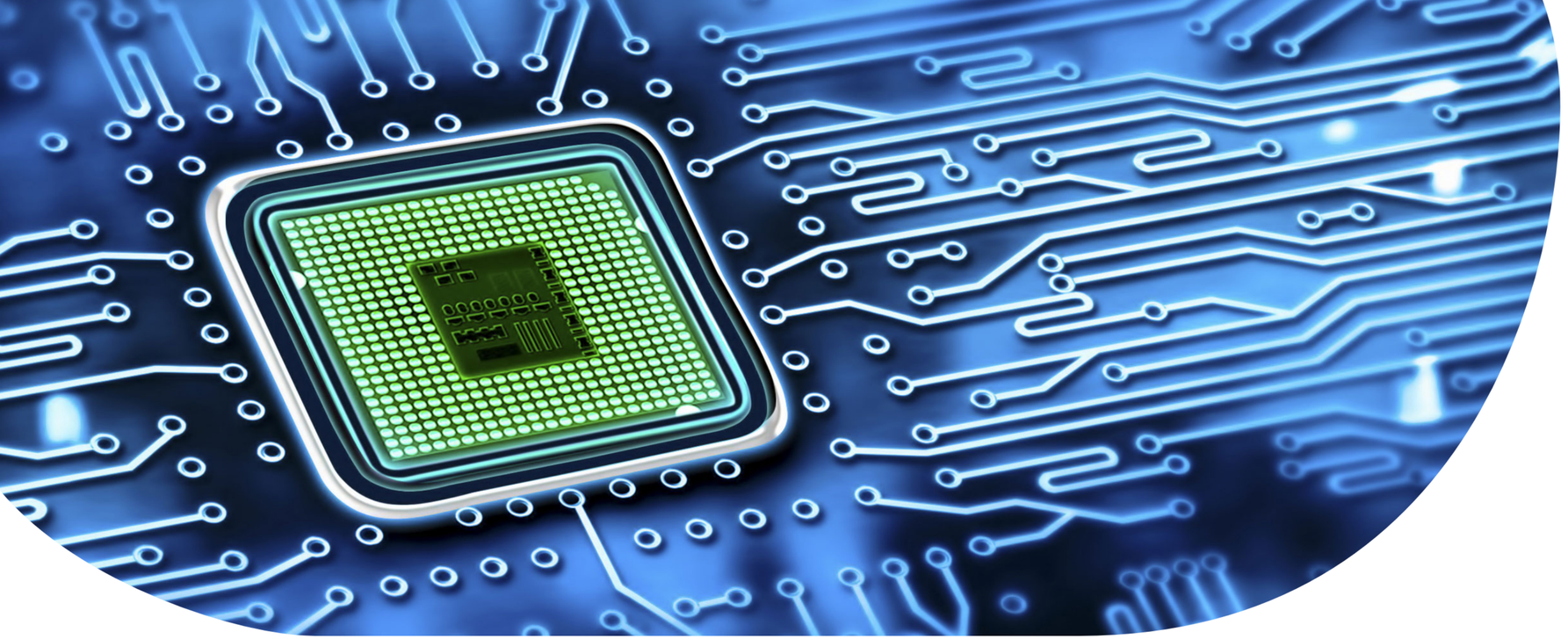


جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

CS256 Digital Design and Computer Architecture

Course Project Outline

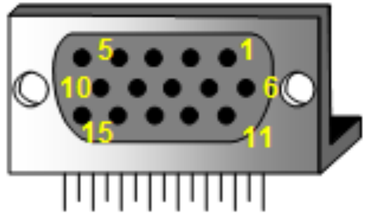
Suhaib Fahmy
suhaib.fahmy@kaust.edu.sa

Course Project Outline

- Aim is to design an arcade game using SystemVerilog, implemented on the FPGA
- Through this project you will learn about **signal timing**, basic **arithmetic**, and have room to show some **creativity**
- You will work in **pairs** on the project
- Your project submission consists of three components (total 40 points):
  - A **individual pre-demo** interview with TA **19th November** *(15 points)*
  - A **group demo** of your game in class on **30th November** *(10 points)*
  - An **individual report** about your design submitted by **10th December** *(15 points)*

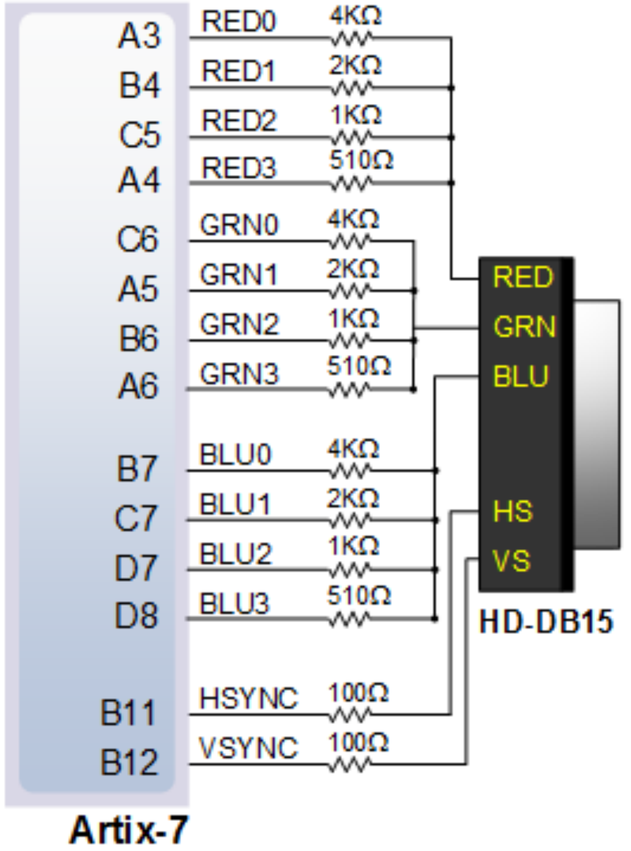


Display via VGA



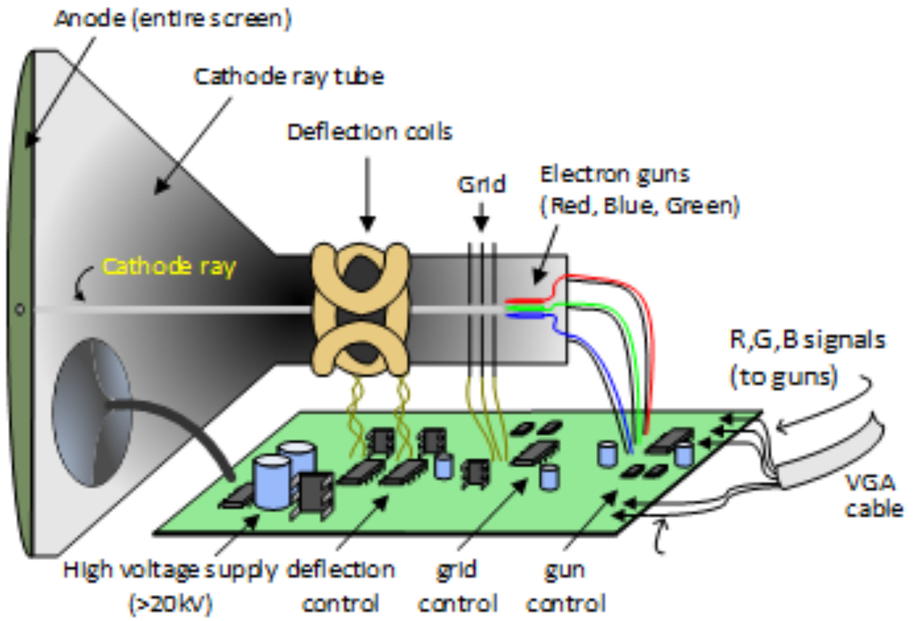
Pin 1: Red
Pin 2: Grn
Pin 3: Blue
Pin 13: HS
Pin 14: VS
Pin 5: GND
Pin 6: Red GND
Pin 7: Grn GND
Pin 8: Blu GND
Pin 10: Sync GND

- VGA is an analog interface that is easy to design hardware for
- The timing principles transfer over to more advanced digital interfaces such as HDMI
- We use VGA as that is what is provided on our development board
- A total of 14 pins are used to drive the VGA output:
  - 4 each for the red, green, and blue values
  - 1 each for horizontal and vertical sync



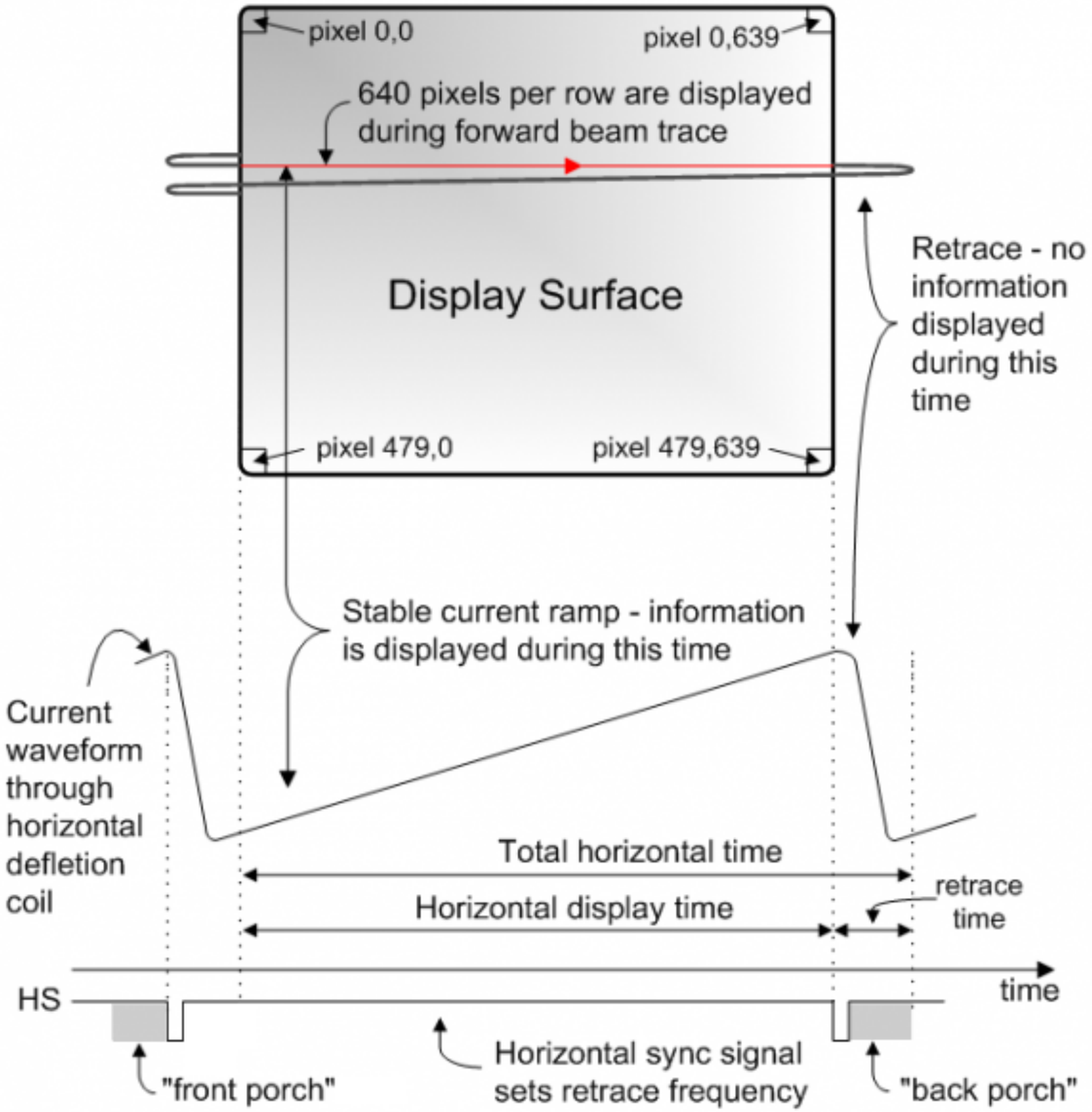
Display via VGA

- VGA was initially designed to control cathode ray tube (CRT) displays
- Electron beams are deflected across a phosphor coated screen where they cause a glow
- The beam is moved in a raster scan pattern, starting from the top left, moving across to the top right, then down by one “row”



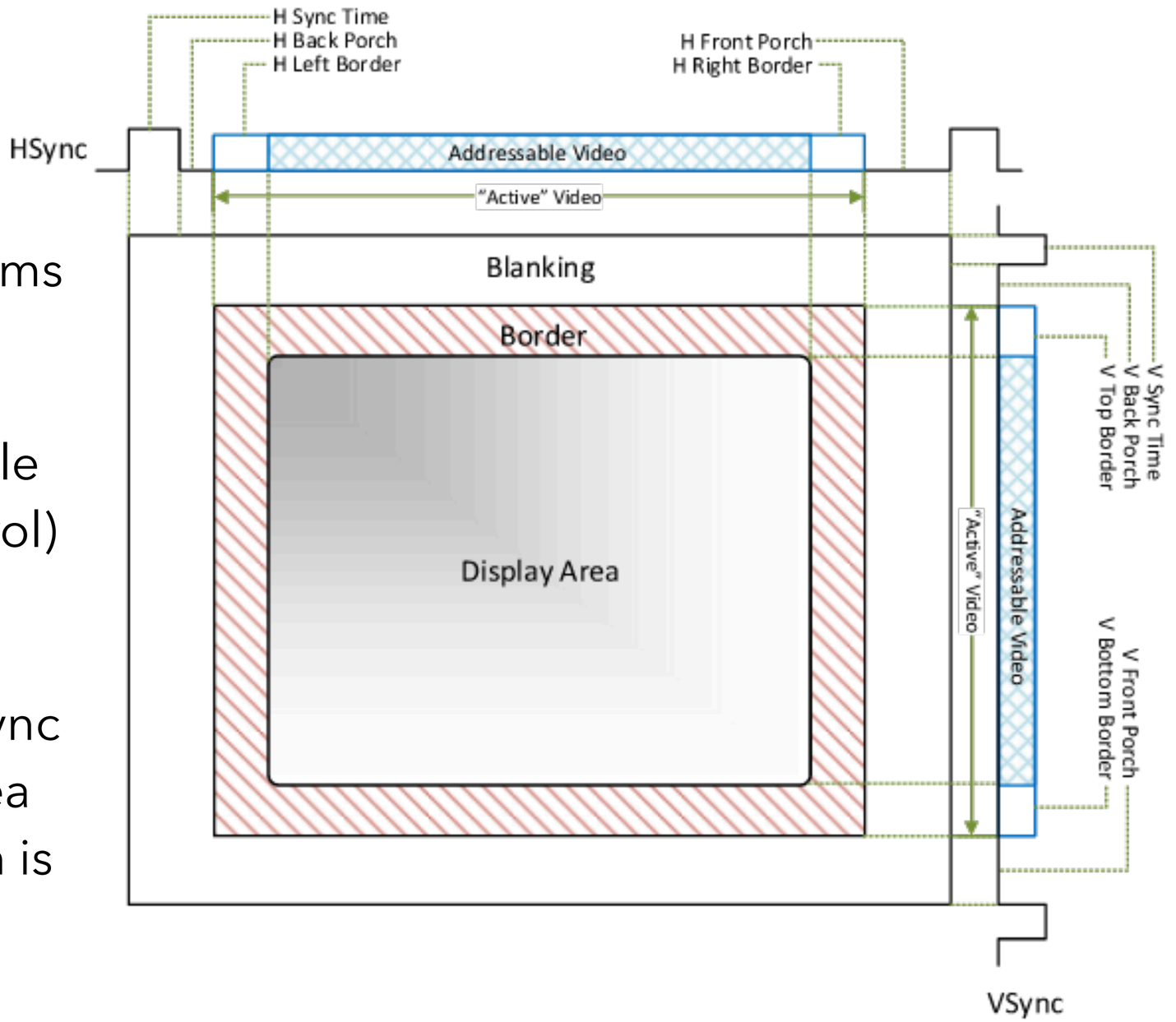
Display via VGA

- Hence, each row is drawn from left to right before moving to the next
- Once the final row is complete, the next frame is started from the top left again
- To move the beam from the end of a row to the start of the next takes time, so there are gaps between rows being drawn



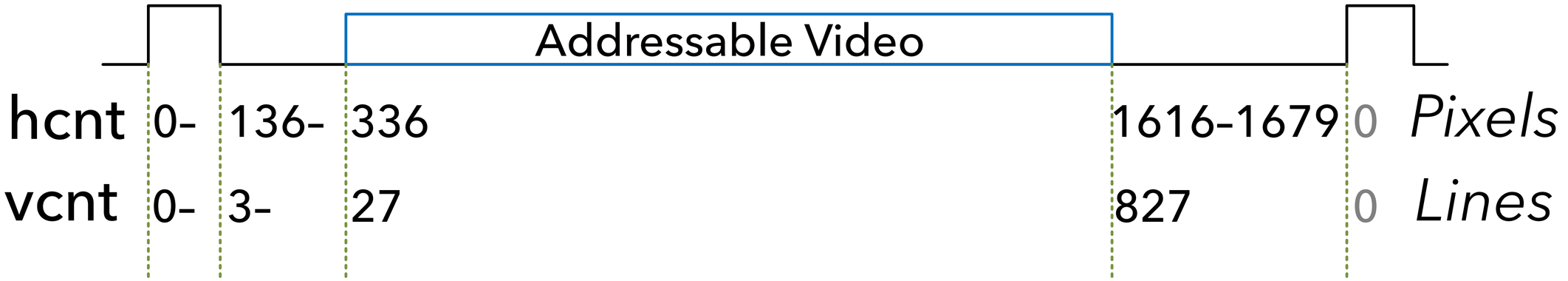
Display via VGA

- Modern LCD displays of course do not involve deflecting electron beams
- However, the timing remains the same for compatibility and to enable other information (e.g. audio, control) to be sent
- The key to understand is that the sync pulses are outside of the drawn area of the screen and the visible region is just a portion of all the signals



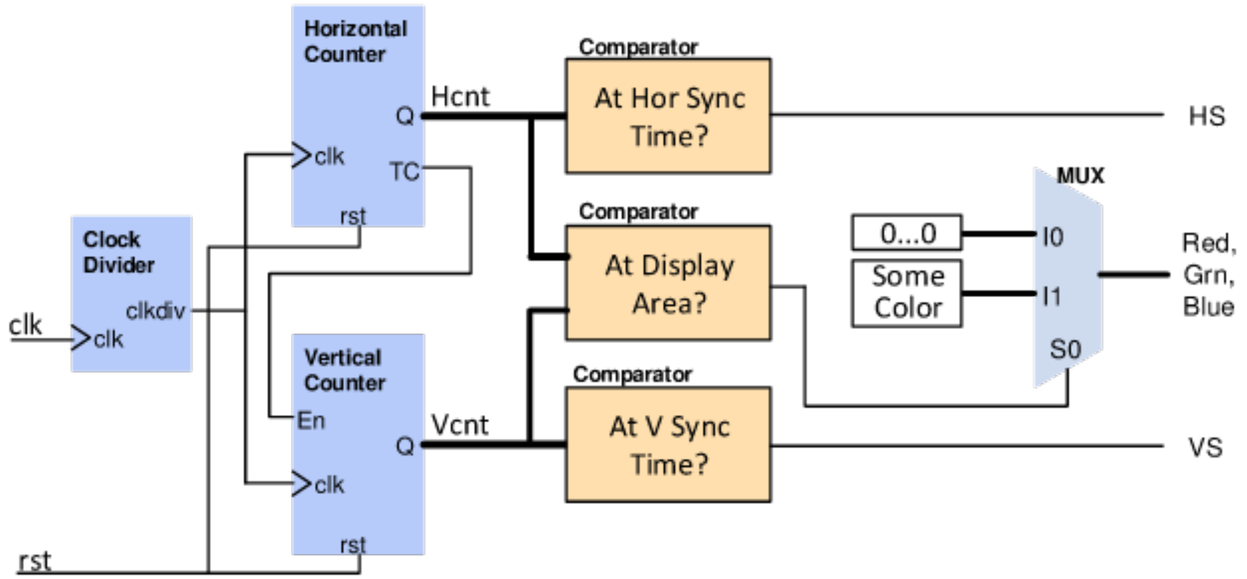
VGA control signals

- The horizontal signals change value in each clock cycle, and must produce pulses as below (for a 1280×800 resolution)
- The vertical signals change only each time we start a new row, and produce longer pulses as below
- The visible video portion is the only time when pixel values are allowed



VGA control signals

- We will use two counters to generate these pulses and output colour to the screen
- The horizontal counter will count on every cycle
- The vertical counter will count whenever the horizontal counter starts again
- The clock should be set to the required frequency



Drawing logic

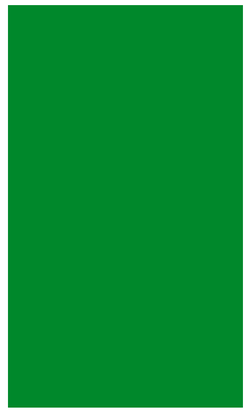
- Now we need additional logic to tell us what pixel in the visible portion we are currently drawing
- We will use additional logic to determine this so at any point, we know what the current (x,y) coordinate is
- Based on what we want to draw on screen, we can use these coordinates to draw a specific colour



AA<y<BB

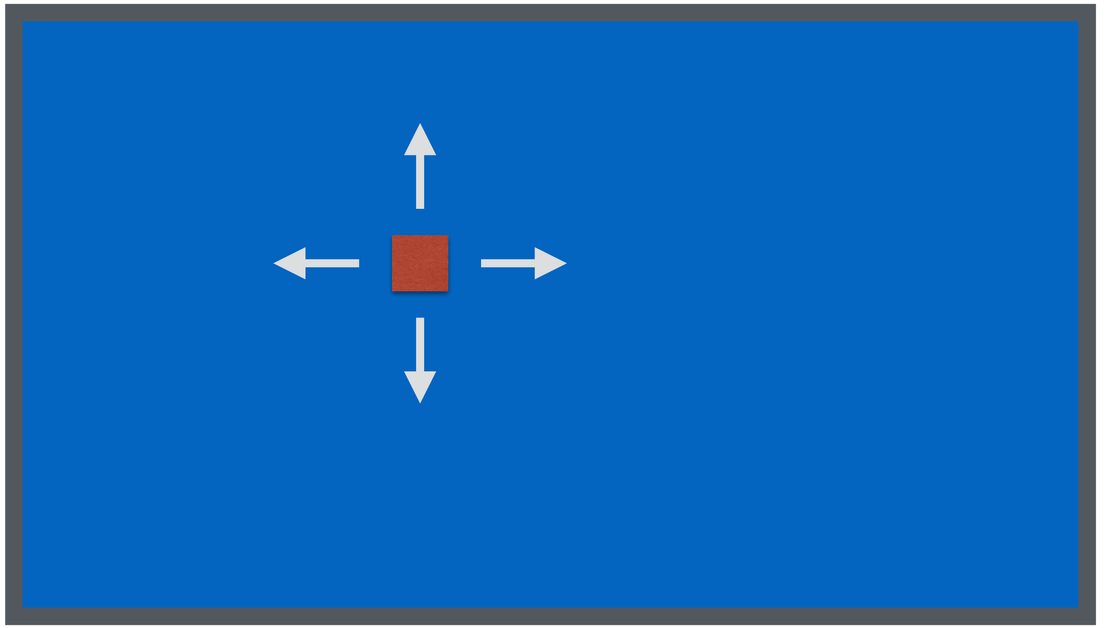
CC<X<DD





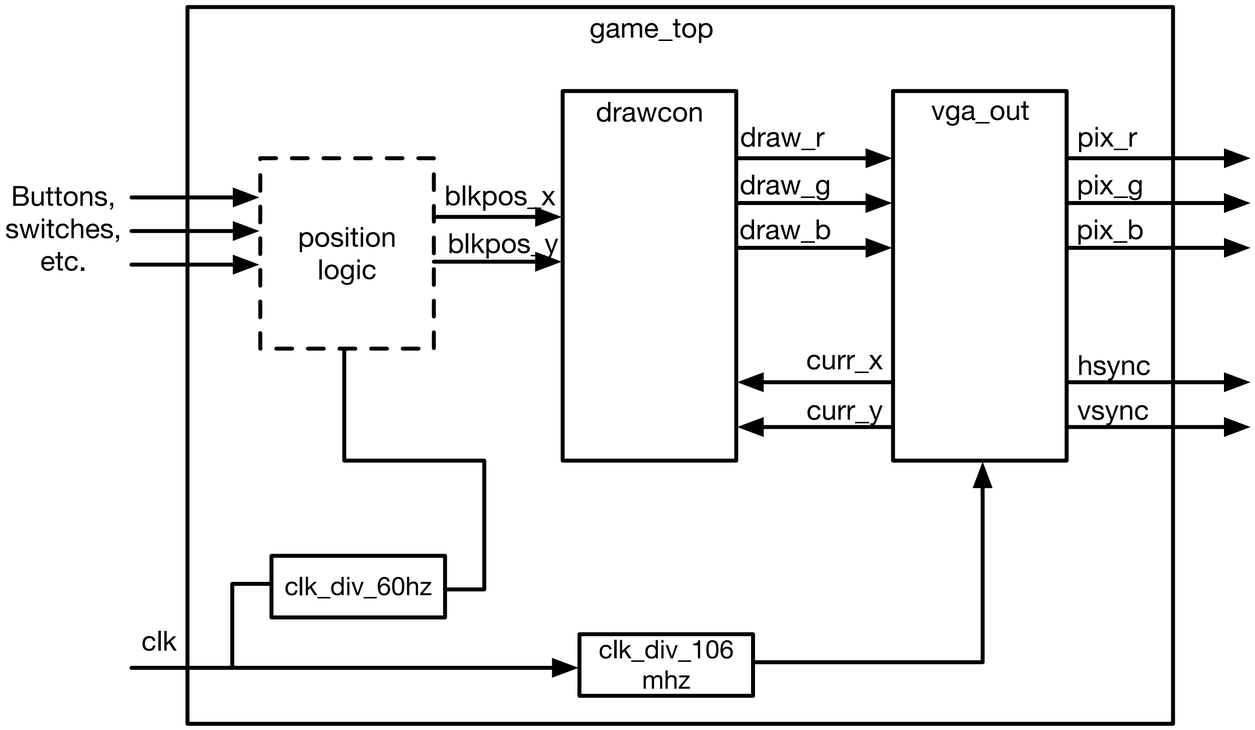
Moving objects

- Now we can modify our design so that x and y coordinates for an object can be checked against the current x and y being drawn
- Buttons can change the values in registers for the object coordinates, allowing it to move
- We can define how movement should interact with other objects, e.g. the borders



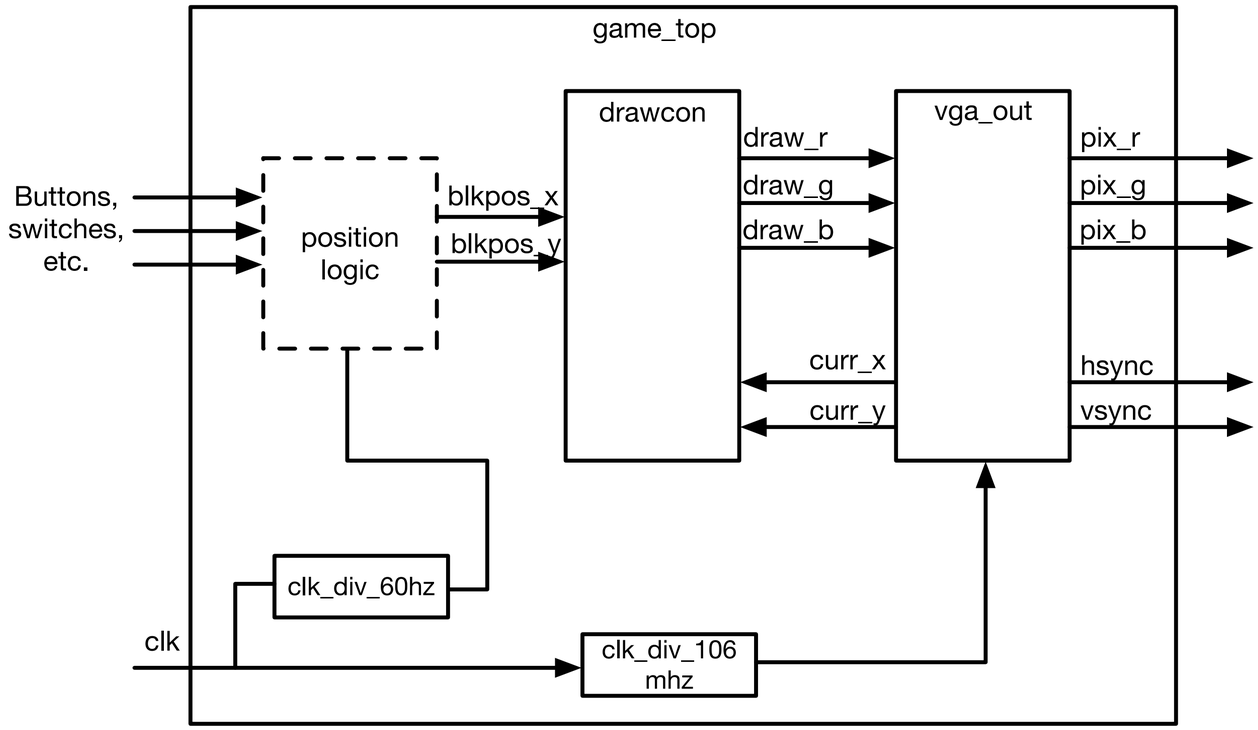
Drawing objects

- At each clock cycle, we are provided with an (x,y) coordinate by the VGA controller
- We must decide what to draw
  - It could be an object if it is in these coordinates
  - It could be a border or wall item
  - It could be the background
- Each of these can change depending on the game



Drawing objects

- This basic template can be used to build a variety of games
- You should start with the basic functionality described before extending
- Consider the I/O you have available to you on the board (buttons, switches, LEDs, seven segment displays)



Possible extensions

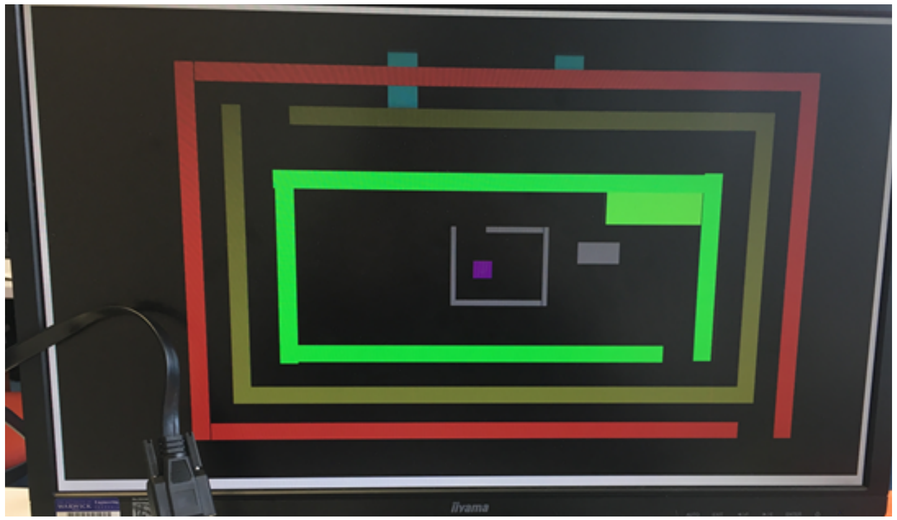
- Consider using sprites for graphics
  - By precomputing a small graphic and storing it in a small memory, you can output different pixel values for parts of an object
- Set up more advanced interactions between objects, e.g. physics based movement
- Use more advanced interfaces such as audio (microphone, speaker) – read the reference manual



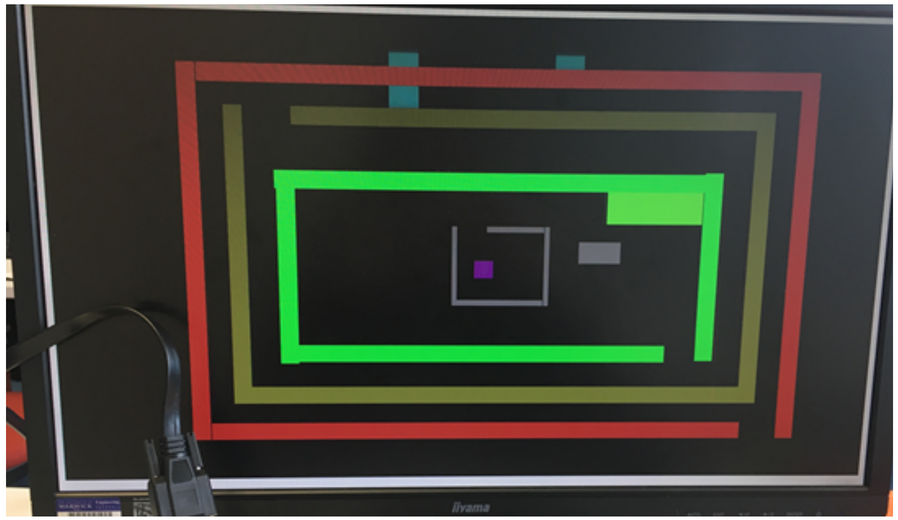
Some examples

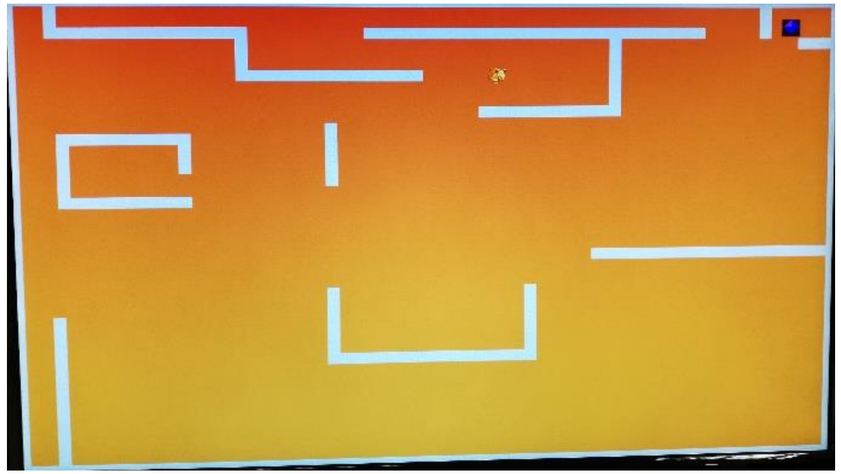


Some examples



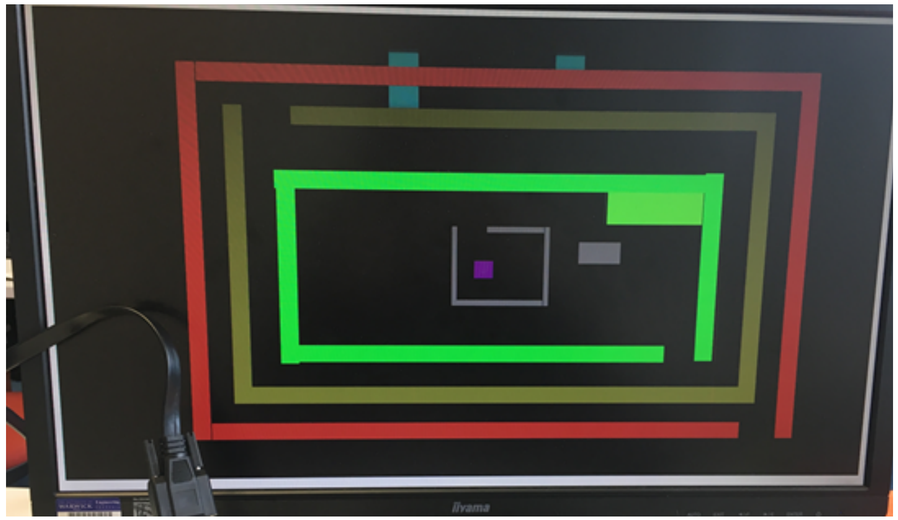
Some examples





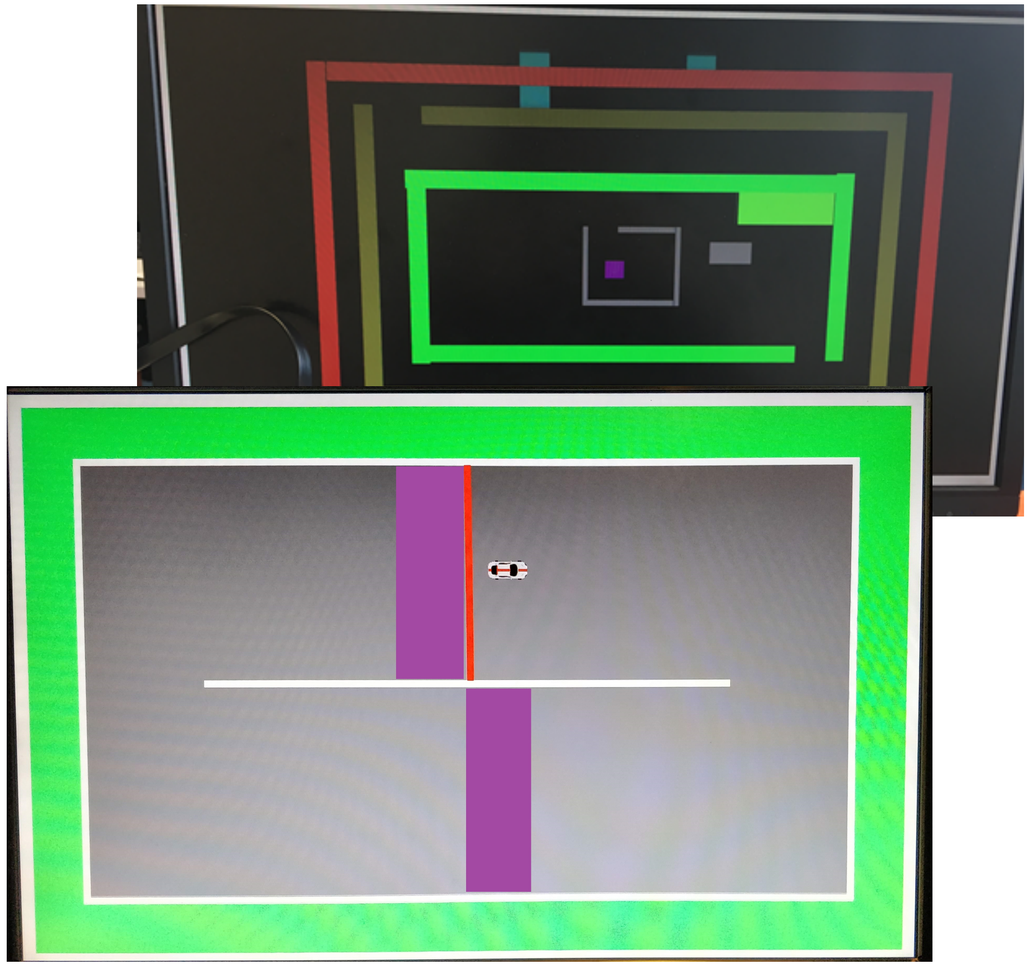
Some examples



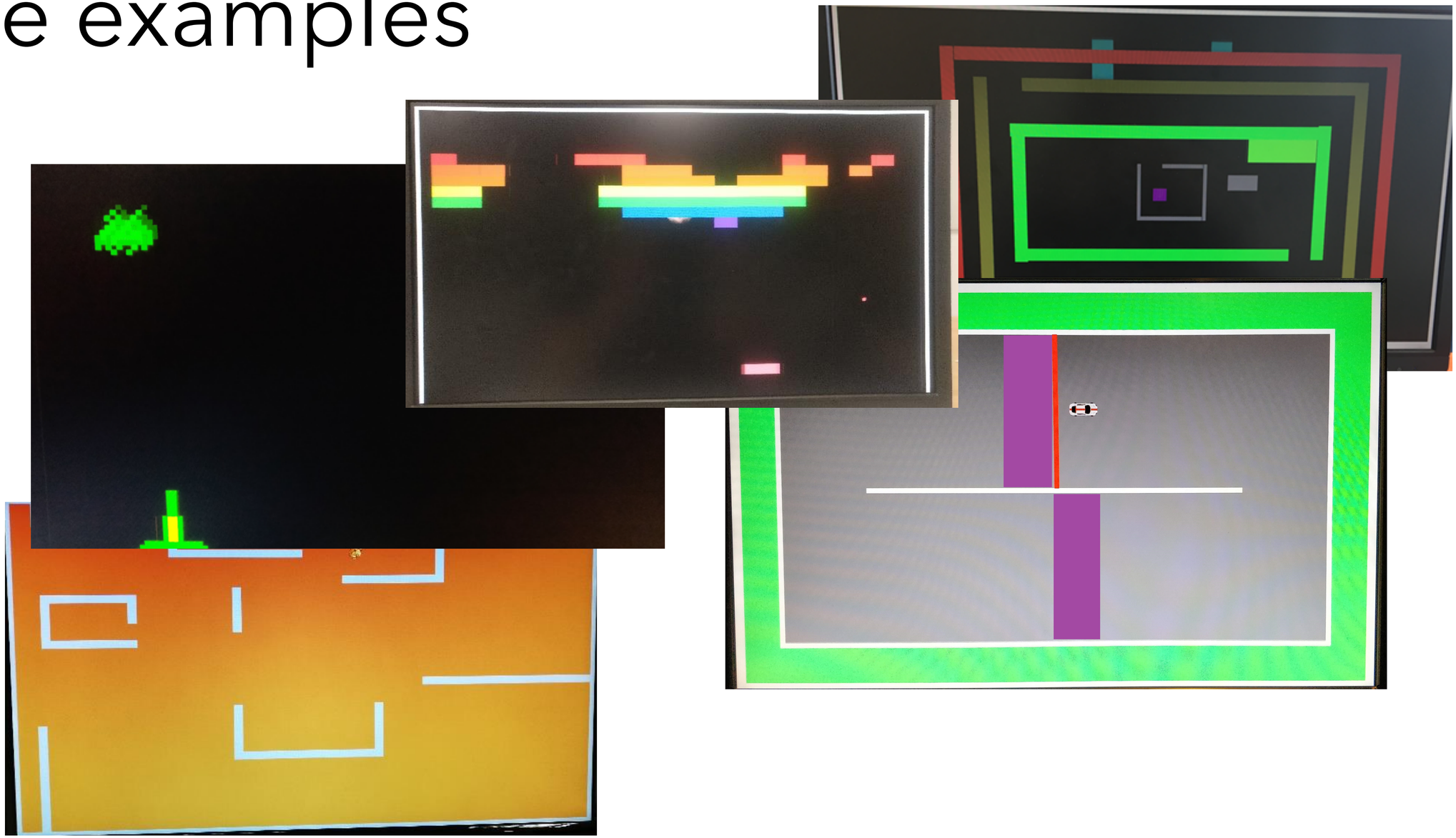


Some examples

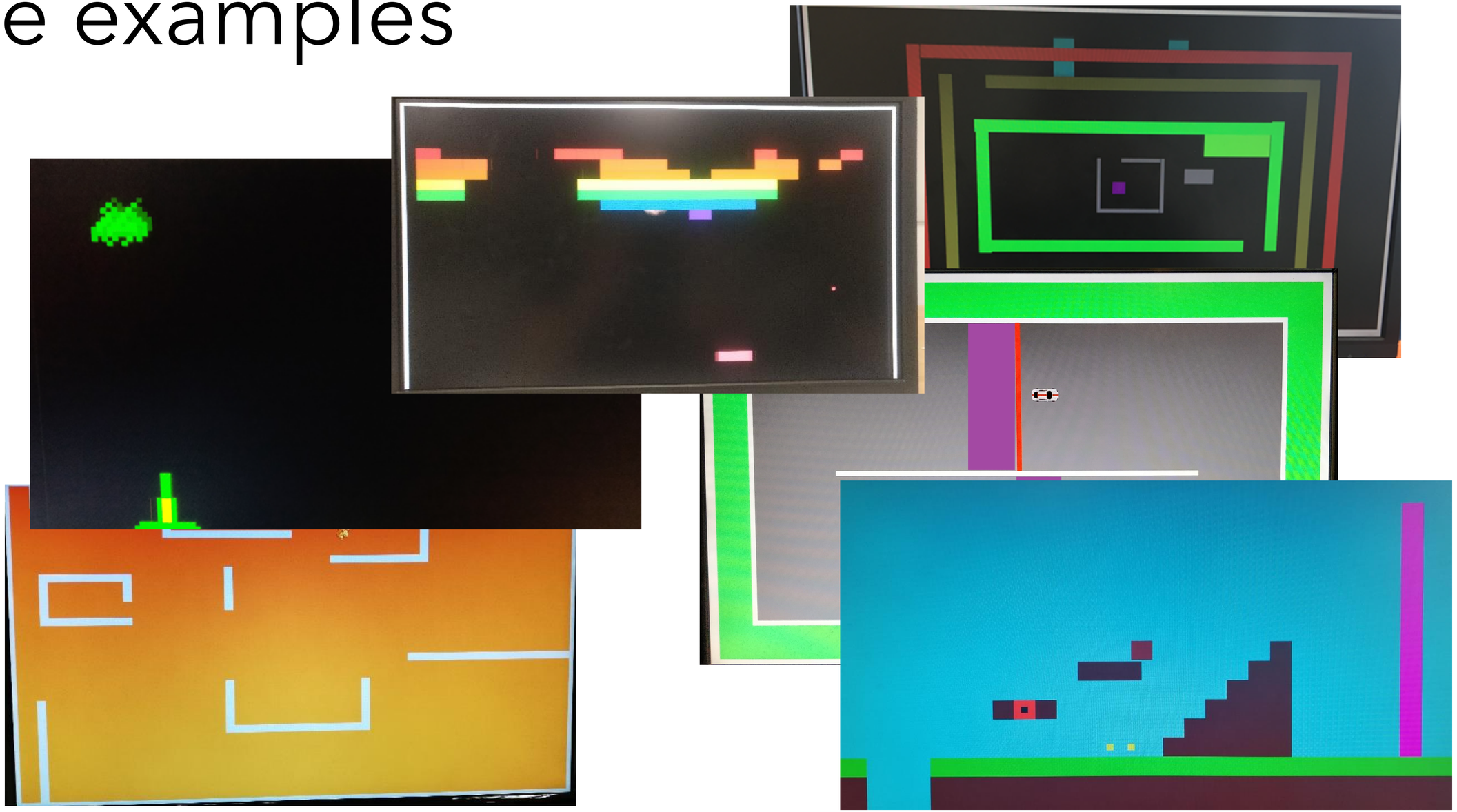




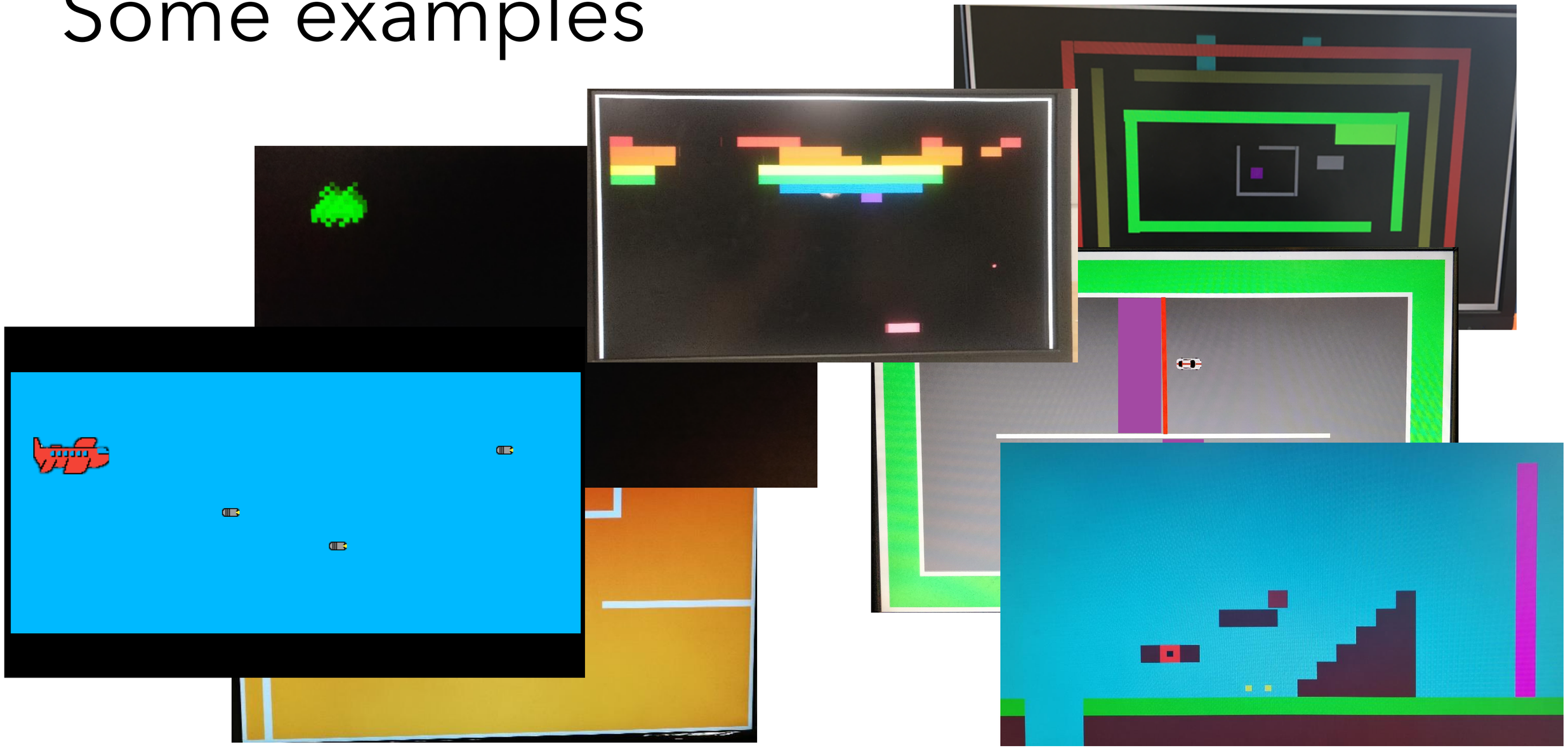
Some examples



Some examples



Some examples



Some examples



Some examples



Some examples



Important considerations

- First focus on getting the VGA output working and tested
- Then focus on position logic for one or more objects
- You should then think about the design of a game that works within the constraints of position based calculations
- Think about how you can use peripherals (score on seven segment display?)
- More advanced features like sprites can be added later



Demo scoring rubric

- Your demo score makes up 10/40 of your score
- Consider the criteria to the right that will be used to mark your demo
- Remember that your individual report and individual pre-demo contributes to final project score

Criteria	Max Score
User control of a moving object on screen with respect for screen boundaries	2.5
A map or multiple objects and interactions between them, e.g. collisions	2.5
Extra features like using other board inputs/outputs	2
Use of sprites from a memory block	2
Marks for creative design ideas	1



More Information

- All details about the project are on the course repository under /classproject/
- There will be milestones to keep track of progress

